CS779 Competition: Machine Translation System for India

Naman Sethi
220688
namansethi22@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

I ranked at the 38th position finally. My final architecture was a transformer based Encoder Decoder Model. My final BLUE Score was 0.160, chrF++ score was 0.367 and ROGUE Score was 0.408. The competition was hectic at times and at a point I got stuck in my Transformer implementations but I am happy to have learned some things from the competition overall.

1 Competition Result

Codalab Username: n_220688
Final Leaderboard Rank on the Test Set: 38
chrF++ Score corresponding to the Final Rank: 0.367
ROUGE Score corresponding to the Final Rank: 0.408
BLEU Score corresponding to the Final Rank: 0.160
Total Number of Submissions in the Training Phase: 28
Total Number of Submissions in the Testing Phase: 8
Table showing the Number of Submissions made each Week during the Training Phase:

Week	Submission Number	Score
1	1	0.09
1	2	0.10
2	1	0.11
2	2	0.14
2	3	0.19
2	4	0.18
3	1	0.16
3	2	0.14
3	3	0.13
3	4	0.21
4	1	0.22
4	2	0.24
4	3	0.30
4	4	0.34

Table 1: Weekly Submissions and Corresponding Scores During the Training Phase.

2 Problem Description

The objective of the task was to make a neural machine translation that is able to effectively translate sentences from english to hindi and bengali. The aim was to try different models, architectures,

pre-processing methods etc.

3 Data Analysis

1. 68849 Bengali sentences are given along with their corresponding English sentences.
2. 80797 Hindi sentences are given along with their corresponding English sentences.
3. Maximum length of sentence in training data: 276
4. Maximum length of sentence in testing data: 176
   Most sentences have 10 -19 token length.

Data	Mean	Median
English Train Tokens	18.47	17.0
Hindi Train Tokens	20.7	18.0
Bengali Train Tokens	15.6	14.0
English Test Tokens	18.19	17.0

Table 2: Token statistics

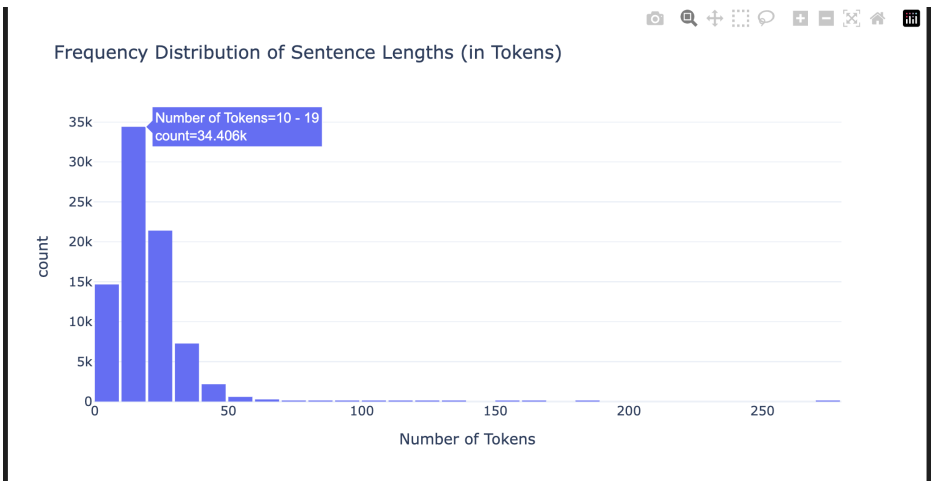

Figure 2: English Sentences Length Freq. Distribution

4 Model Description

I have majorly worked on GRU, LSTM and Transformer based Encoder Decoder Models. I have divided the whole journey into 3 phases:

Phase 1: Baseline Experiments with GRU and LSTM Architectures

In the initial part of the competition I first tried to work with the given GRU based Encoder Decoder Model. Since I had some prior experience of working with LSTMs I also wanted to try them out. So in the initial phase I was mostly playing around with these 2 architectures. Both models were trained for 5 epochs with randomly initialized embeddings at first. These were some of the results that I had obtained through these models. As you can see there are clear repetitions of words and lack of identification and transaltion of proper nouns in these sentences. This could be considered as a phase where baseline models were being established.

505511	इस पर , मैं क्या कर रहा हूं ?
505512	श्वास झुकते और बचें और बचें ।
505513	उपयोग : जौ के उपयोग उपयोग के लिए उपयोग किया जाता है , हालांकि , के लिए उपयोग
505514	इस बार - बार बार - बार बार के लिए एक बार तक लिए एक बार ।
505515	यहाँ राष्ट्रीय उद्यान के सबसे एक में एक है , जो कि के के के में से एक

Figure 3: Some Example Translations in this phase

Phase 2: Training Optimization and Incorporation of Pretrained Embeddings

In the second phase, I experimented with Word2Vec embeddings (from Google News) and GloVe embeddings, both of which significantly improved the model's performance by providing richer semantic representations compared to random initialization. I gradually increased the number of training epochs and carefully tuned the sequence padding and normalization of Indic text to handle variable-length inputs effectively. The key takeaway from this phase was that pretrained embeddings and careful text normalization play a crucial role in improving cross-lingual understanding, and that model performance tends to saturate after a certain number of epochs unless more sophisticated architectures or features are introduced.

Phase 3: Transition to Transformer-Based Encoder–Decoder Models

Transformers were also something that interested me and I had heard they were sota models for these kind of tasks. So after results began to get saturated from the above models, I switched to Transformer based encoder decoder models. There was a lot of playing around possible with these models.Initially, the results were suboptimal due to misalignment between Hindi and English embeddings, as the model struggled to learn cross-lingual mappings effectively. To address this, I introduced a CNN-based alignment layer that projected embeddings from both languages into a shared vector space before feeding them into the Transformer. This modification led to a noticeable improvement in model accuracy and stability.

Final Model

1. The final model architecture is inspired by the Transformer model introduced in "Attention Is All You Need" (Vaswani et al., 2017), with further enhancements to handle morphologically rich languages like Hindi.
2. Specifically, I integrated character-level CNN embeddings (inspired by Kim et al., 2016: Character-Aware Neural Language Models) alongside pretrained word embeddings, allowing the model to capture subword information effectively.
3. The final architecture is a hybrid sequence-to-sequence Transformer combining word embeddings initialized from word2vec vectors for English.
4. A Character-level CNN encoder that extracts morphological features from token-level character sequences has been used. The character embeddings are concatenated with word embeddings and projected into the Transformer model dimension before being passed through the encoder or decoder.
5. A 4-layer encoder–decoder Transformer with: Model dimension =300, 4 attention heads per layer, Dropout rate of 0.1 .

   The encoder processes the source sequence (English) with positional encoding and self-attention layers, while the decoder attends to both target-side context and encoder outputs. I also included a bridge layer (a linear projection) to align encoder–decoder latent spaces for better stability. .

   I have used the Cross-Entropy Loss with label smoothing ($\varepsilon = 0.1$) to prevent overconfidence and improve generalization. Padding tokens are ignored during loss computation. The loss function is defined as:

$$\mathcal{L} = -\sum_t y_t \log \hat{y}_t$$

where y_t is the target token (ground-truth distribution) and $\hat{y}_t$ is the predicted probability from the decoder's softmax layer.

Validation

I experimented with both greedy decoding and (very late in the competition) beam search: Greedy decoding was used in the final submission was almost all the duration of the training phase. I tried beam search in the test phase, but it was too late to make a difference.

On the final test data set, the submission was made using greedy decoding due to time constraints and repeated crashes of kaggle. Prelimnary analysis of sentences produced by beam search (width =5) did indicate that it was a better method of validation. The sentences had better semantics and lesser word repetitions. An observation was that for most of the sentences greedy and beam search was translating into the same sentence. However, for longer sentences beam search was producing marginally better results.

5 Experiments

Data Preprocessing

(a) **Tokenization:** I used `nltk.word_tokenize` to split English sentences into tokens. This step converts raw text into a list of words, which helps the model treat each token as an independent input unit rather than an entire sentence string.

(b) **Target Language Normalization:** For Hindi and Bengali, I used the IndicNLP normalizer to standardize script representations and remove inconsistencies. Normalization ensures that equivalent characters are represented uniformly, preventing vocabulary explosion caused by orthographic variations. This improves embedding consistency and helps the model generalize better.

(c) **Vocabulary Encoding and Padding:** I mapped each token to a unique integer index using the constructed vocabularies (`en_word2index`, `de_word2index`) and padded all sequences to a fixed length (40 tokens) using special symbols: `SOS`, `EOS`, and `PAD`. This was done using the `encode_and_pad()` function. Neural models require fixed-length input tensors for efficient batching. `SOS` and `EOS` markers help the model learn sequence boundaries. Padding ensures shorter sentences don't affect positional alignment in batch training.

5.1 Training procedure

Optimization is performed using the **AdamW** optimizer with discriminative learning rates:

$$EncoderLR = 1 \times 10^{-4}, \quad DecoderLR = 3 \times 10^{-4}, \quad WeightDecay = 1 \times 10^{-4}$$

The final model was trained at 8 epochs. I tested the model at 5 ,8 and 10 epochs and it showed best performance at 8 epochs among the 3. It had a tendency to overfit at 10 epochs.

For the final model, using T4 GPU the training time was around 15-20 minutes per epoch. Initially I was training and testing my models at sentences that were padded to 20 tokens, in order to check out different models in lesser time. At that time one epoch took less than 5 minutes to train. On increasing the sequence length to 40, there was a huge increase in model training time.

On the other hand the LSTM and GRU based encoder decoder models were much faster to train. I had tested them upto 10 epochs and they performed best at these many epochs. I couldn't look into higher epochs for these models becuase gains from increasing epochs were low and I wanted to try other models in the mean time as well. Taking constraints on computing resources in mind, I had to look past these models and finally move onto the Transformer Model.

6 Results

6.1 Major Models

Model Architecture	**Other details**	**Score**
LSTM based Encoder,Decoder	Trained embeddings from scratch, 5 epochs	0.09
GRU based Encoder,Decoder	Trained embeddings from scratch, 5 epochs	0.10
LSTM based Encoder,Decoder	Trained embeddings from scratch, 7 epochs	0.14
GRU based Encoder,Decoder	Trained embeddings from scratch, 7 epochs	0.18
LSTM based Encoder,Decoder	Pre-trained embeddings(glove), 7 epochs	0.18
LSTM based Encoder,Decoder	Pre-trained embeddings(word2vec), 7 epochs	0.19
Transformer based Encoder,Decoder	Pre-trained embeddings(word2vec), 5 epochs	0.24
Transformer based Encoder,Decoder	Pre-trained embeddings(word2vec), 8 epochs	0.3
Transformer based Encoder,Decoder	Added CNN layer to above, 8 epochs	0.34

6.2 Results

The best-performing model was the Transformer-based Encoder–Decoder with pre-trained Word2Vec embeddings and a CNN layer, trained for 8 epochs achieving a score of 0.34. This model outperformed all recurrent (LSTM/GRU) architectures. Pre-trained Word2Vec embeddings provided good starting point for the embeddings. The Transformer's self-attention mechanism effectively captured long-range dependencies and contextual relationships across sentences, unlike sequential LSTMs/GRUs. Training for more epochs allowed the model to fully leverage the attention-based representations for better alignment between source and target sequences.

7 Error Analysis

7.1 Error across models

(a) Across all experiments, the Transformer-based models performed significantly better than the RNN-based models on both the development and test sets. The LSTM and GRU models trained with random embeddings struggled with long and complex sentences, often producing truncated or repetitive outputs.

(b) When pre-trained embeddings such as Word2Vec or GloVe were introduced, the LSTM and GRU models showed moderate improvement in accuracy and word choice, as the embeddings provided better semantic grounding. However, their translations were still not good enough.

(c) In contrast, the Transformer models with pre-trained Word2Vec embeddings demonstrated a clear improvement in both accuracy and fluency. The self-attention mechanism allowed the model to capture global dependencies across the input sequence, leading to more appropriate translations. These models achieved higher BLEU scores compared to their RNN counterparts.

(d) The primary source of errors in the Transformer models was minor repetition or over-reliance on frequent target language tokens. The best-performing model overall was the Transformer-based encoder-decoder with pre-trained embeddings, achieving the highest BLEU score. Its ability to model long-distance relationships and use pre-trained embeddings for richer contextual understanding made it more robust and accurate. Adding a CNN-based character embedding layer further improved handling of out-of-vocabulary and complex words. Overall, the Transformer architecture proved to be the most effective and stable approach for this multilingual translation task.

7.2 Model Imperfections

The final Transformer model integrated with a character-level CNN showed strong improvements over all previous architectures, but it still exhibited certain weaknesses that explain

why it was not perfect. The inclusion of the CNN layer significantly improved the model's handling of morphologically rich words and rare tokens. This character-level processing helped the model capture subword information and handle minor spelling variations better than purely word-level Transformers. However, some challenges persisted.
One major limitation was in handling rare proper nouns and named entities — especially person names, regional terms, and organization titles. Even with character-level embeddings, the model sometimes replaced these with phonetically similar but semantically incorrect words, or occasionally left them untranslated.
Another recurring error pattern was token repetition in longer sentences, where the decoder would generate repeated words or phrases. This could be improved using other validation techniques apart from greedy like beam search.

7.3 Interesting Insights

An interesting insight was that the char-level CNN enhanced robustness to noisy or informal text — for example, words with slight spelling variations, regional spellings, or missing diacritics were handled in a better manner. This suggests that combining subword-level and character-level features yields a richer and more flexible representation, especially beneficial for linguistically complex languages like Hindi and Bengali.

8 Conclusion

1. Key Findings: The Transformer model with pre-trained word embeddings and integrated character-level CNN achieved the best overall performance, effectively capturing morphological and subword patterns in Hindi and Bengali. It outperformed recurrent models and pure Transformers without CNNs, especially in handling inflectional variations and noisy inputs. However, challenges remain with rare proper nouns, long-distance dependencies, and idiomatic expressions. Recommendations Future Work:
2. Future improvements would certainly include testing other methods of validation like dp and beam search. Also, some hyperparameters in preprocessing such as sequence length of padding could be fine tuned as well. Additionally, if pre trained embeddings for Hindi could be used, that should be a great boost to the model. Lastly, finding morphemes while tokenizing should also improve the model considerably.